HyperVector-Optimizer: Correlation-Aware Selectivity Estimation and Hardware-Calibrated Plan Routing for Filtered Vector Search

Sanjay Mishra
Independent Researcher (IEEE Senior Member)
Raleigh, NC, USA
sanmish4@icloud.com

Divya Chukkapalli
Independent Researcher (IEEE Senior Member)
Raleigh, NC, USA
divya.95j@gmail.com

Abstract

Filtered top-k vector search—a nearest-neighbor query restricted by relational predicates and foreign-key semi-joins—can be executed by pre-filtering and scanning the survivors, by filtering inline during graph traversal, or by post-filtering an unfiltered graph search. Which plan is fastest depends less on the predicate's global selectivity than on its *local* selectivity around the query vector, and engines that route on global selectivity misroute exactly the correlated queries that matter. We present *HyperVector-Optimizer* (HVO), an optimizer with three parts. (i) The *Polytope-Ball Sketch* (HPBS) partitions the vector space into k-means cells that carry radial distance profiles, per-attribute histograms, and row-weighted foreign-key samples, and estimates local selectivity as k divided by the size of the smallest ball that holds k estimated matches; a 497-parameter calibrator corrects its residual bias. (ii) A cost model calibrated on the target machine captures the doubling search budgets of adaptive graph plans, their superlinear per-round cost, and the scan kernel's SIMD width. (iii) A minimax router picks the plan whose worst-case cost over the estimator's calibrated uncertainty interval is smallest. We prove a sensitivity lemma for doubling plans, regret bounds for point and minimax routing (the latter has zero regret whenever the exactly costed pre-filter plan is optimal), and a lower bound showing that, when predicates are evaluated through an index, any router using only global selectivity has regret that grows polynomially in the table size. We evaluate on one million arXiv papers with MiniLM embeddings and real metadata, and on two synthetic million-row benchmarks with controlled correlation (128-d and 768-d), measuring every plan single-threaded on an Apple M4 with faiss HNSW. On the real data, HPBS reaches a P99 local-selectivity Q-error of 6.17, against 39.23 for HNSW index probing, 18.41 for sampling, and 765.57 for independence-based histograms. HVO lowers mean latency by 65% relative to the best tuned global-selectivity router (1.84 vs. 5.25 ms; an oracle that runs every plan reaches 1.53 ms), and its P99 latency (13.95 ms) is close to the oracle's (13.67 ms). On the synthetic benchmarks the mean-latency gains are 35% and 23%, but there HVO does not improve P99 latency over the simplest threshold rule. We release the code and every experiment.

Keywords

Vector databases, filtered approximate nearest neighbor search, cardinality estimation, query optimization, HNSW, SIMD, regret bounds

1 Introduction

Retrieval-augmented generation, recommendation, and analytical workloads increasingly issue *filtered* vector queries: a top-k similarity search over embeddings restricted by relational predicates and joins [1–4]. A typical query is

$$\begin{aligned}&\texttt{SELECT } r.\texttt{id FROM } R\ r \texttt{ JOIN } S\ s \texttt{ ON } r.\texttt{fk} = s.\texttt{pk}\\&\texttt{WHERE } r.a_1 \in [l_1, u_1] \wedge r.a_2 \in C \wedge s.a_3 \in D\\&\texttt{ORDER BY } \operatorname{dist}(r.\mathbf{v}, \mathbf{q}) \texttt{ LIMIT } k.\end{aligned} \tag{1}$$

General-purpose engines expose vector indexes inside the relational executor (PostgreSQL's `pgvector` [8], DuckDB's `vss` [9]), and specialized systems attach predicate bitmaps to graph and partition indexes (Milvus [10]). Whatever the engine, three physical plans are available:

(1) *Pre-filtering* (Π_{pre}): evaluate the predicate, then scan the $s_g N$ qualifying vectors with a SIMD kernel. Exact, and its cost depends only on the global selectivity s_g.
(2) *Post-filtering* (Π_{post}): search the unfiltered HNSW graph [11] and verify the predicate on candidates, enlarging the candidate budget until k matches are found (as in `pgvector`'s iterative scan and VBase [2]).
(3) *Fused traversal* (Π_{fused}): materialize the predicate bitmap and filter inline during graph traversal [3, 10].

The cost of the two graph plans is governed by how far the graph search must travel before it meets k matching tuples. That distance depends on the *local* selectivity s_{loc} of the predicate near the query, not on s_g. When the filter is anti-correlated with the query's neighborhood—a machine-learning query vector filtered to `category = 'Medieval History'`—a moderate s_g hides a tiny s_{loc} and post-filtering traverses a large part of the graph. When the filter is positively correlated, a small s_g hides a large s_{loc} and pre-filtering scans many vectors that post-filtering would never touch.

The problem. Optimizers today route on global selectivity, estimated with one-dimensional histograms under attribute-value independence [19], or on an exact predicate count that ignores the vector. Recent work recognizes that this fails: Exqutor estimates vector-search cardinality inside the optimizer by probing the index or sampling [18], and benchmark studies document large performance swings under correlated filters [4, 6, 7]. Three questions remain open: how to estimate local selectivity cheaply without touching the index during planning, how to model plan cost on the actual hardware, and what can be guaranteed about routing when the estimates are wrong.

Our approach. HVO answers each question with a component, and each component comes with an analysis and a measurement:

- **HPBS, a polytope-ball sketch (Section 4).** We define local selectivity operationally, as k/h_k where h_k is the number of tuples at least as close to $\mathbf{q}$ as its k-th matching tuple. This is exactly the horizon a post-filtering plan must cover. HPBS estimates h_k from L partitions of m k-means cells, each carrying a radial distance profile, per-attribute histograms, and a row-weighted bottom-k sample of foreign keys. A bisection finds the radius whose estimated ball holds k matches. A deterministic error decomposition (Proposition 4.1) separates geometric from within-cell error. The query path is C, taking 85–147 μs, and never touches the vector index.
- **A hardware-calibrated cost model (Section 5).** Graph plans double their budget, and on our machine a graph round costs superlinearly more as its budget grows (fitted exponent $\gamma = 1.70$ for post-filtering). We model plans as cost *staircases*, calibrate them on held-out queries, and include the scan kernel (scalar FP32, NEON FP32, NEON INT8), which shifts the plan boundary by the ratio of kernel speeds (Theorem 5.1).
- **Regret analysis and minimax routing (Section 6).** We bound how much a doubling plan's cost can change under a multiplicative estimation error (Lemma 6.1), derive regret bounds for point routing (Theorem 6.2), and introduce minimax routing over a calibrated interval, which has zero regret whenever the exactly costed pre-filter plan is optimal (Theorem 6.3). Any router that sees only s_g has worst-case regret $\Omega(N^{\gamma/2})$ when predicates are evaluated through an index (Proposition 6.4).
- **An end-to-end evaluation in which every plan runs (Section 7).** On three one-million-row benchmarks—arXiv papers with real metadata, and two synthetic datasets with controlled correlation—we execute all three plans for every test query, which gives an exact oracle. Against it we compare static plans, tuned global-selectivity routers, and our cost model fed by histograms, exact counts, sampling, index probing, and HPBS.

Scope. We evaluate on one real and two synthetic datasets, on one machine, with plans implemented on faiss [12] rather than inside a production DBMS. Section 8 discusses what this does and does not establish.

2 Background and Related Work

Hybrid execution engines. Relational engines with vector support include AnalyticDB-V [1], VBase [2], which iterates HNSW results under relaxed monotonicity until predicates are satisfied, `pgvector` with iterative index scans [8], and DuckDB `vss` [9]. Milvus [10] evaluates scalar predicates into a bitset that the vector kernel consults during search. These systems choose among plans with rules or with cost models that do not see the joint distribution of predicate and query vector.

Filtered ANN indexes. A second line of work bakes attributes into the index. Filtered-DiskANN [13] and NHQ [14] incorporate labels into graph construction; HQANN [15] fuses attribute and vector distances; SeRF [16] and its dynamic successor [17] build segment graphs for one-dimensional range filters; ACORN [3] densifies HNSW and expands two-hop neighborhoods to stay connected under arbitrary predicates. Surveys and benchmarks [4–7] find that no index dominates across selectivities. Our work is complementary: whichever indexes exist, the optimizer still has to choose a plan.

Cardinality estimation. Relational estimators range from histograms and wavelets [20, 21] through sketches such as HyperLogLog and k-minimum values [22, 23] to learned models [24–26]. For similarity queries, estimators use LSH [27, 28] or reference objects [29]. Exqutor [18] is closest to us: it plugs vector-search cardinality estimates into a relational optimizer, using index-based techniques when an HNSW or IVF index exists and adaptive sampling otherwise. HPBS differs in three ways. It estimates the *joint* quantity (matches inside a ball) from a synopsis without probing the index; it targets the k-th-match horizon that determines graph-plan cost; and we pair it with a hardware-calibrated cost model and regret analysis. We compare against an Exqutor-style index probe and a sampling estimator; we do not reproduce Exqutor itself.

Notation

R has N tuples with unit-norm vectors $\mathbf{v}_i \in \mathbb{R}^d$ and attributes $\mathbf{a}_i$; a query is $(\mathbf{q}, k, \mathcal{P})$, where $\mathcal{P}$ is a conjunction of range, `IN`, and semi-join predicates. $V(\mathcal{P})$ is the set of matching tuples, $s_g = |V(\mathcal{P})|/N$, and the Q-error of an estimate is $\max(\hat{c}/c, c/\hat{c})$.

3 Problem Formulation

Definition 3.1 (k-th-match local selectivity). Order R by distance to $\mathbf{q}$. Let $h_k(\mathbf{q}, \mathcal{P})$ be the number of tuples whose distance to $\mathbf{q}$ is at most that of the k-th closest tuple in $V(\mathcal{P})$. The local selectivity is $s_{\text{loc}}(\mathbf{q}, \mathcal{P}) = k/h_k(\mathbf{q}, \mathcal{P})$.

Equivalently, s_{loc} is the fraction of matches inside the smallest ball around $\mathbf{q}$ that contains k matches. Under independence between $\mathcal{P}$ and the vectors, $h_k \approx k/s_g$ and $s_{\text{loc}} \approx s_g$. An earlier version of this work measured the match fraction inside a ball of fixed size k/s_g. That quantity predicted graph-plan cost poorly under anti-correlation: there the k-th match lies far outside that ball, so the plan must search much further than the fixed ball suggests. Definition 3.1 measures that distance directly. We quantify correlation by the global-local selectivity coefficient [4]

$$\rho_{\text{GLS}} = \tanh\left(\tfrac{1}{2} \ln \frac{s_{\text{loc}}(1 - s_g)}{s_g(1 - s_{\text{loc}})}\right) \in (-1, 1), \tag{2}$$

and call a query anti-correlated if $\rho_{\text{GLS}} < -0.3$, positively correlated if $\rho_{\text{GLS}} > 0.3$, and independent otherwise.

The optimizer must choose $\hat{\Pi} \in \{\Pi_{\text{pre}}, \Pi_{\text{fused}}, \Pi_{\text{post}}\}$ from estimates $(\hat{s}_g, \hat{s}_{\text{loc}})$ produced in planning time that is small relative to execution. Figure 1 shows the architecture.

4 The Polytope-Ball Sketch

HPBS answers the question *"how many tuples lie in the ball around* $\mathbf{q}$ *that holds* k *matches of* $\mathcal{P}$*?"* The ball is a region in $\mathbb{R}^d$; the predicate is an axis-aligned polytope in attribute space joined through foreign keys. Space-partitioning trees degrade to scans for $d \gtrsim 30$, and the predicate is not known until query time, so we summarize both sides per cell of a coarse vector partition.

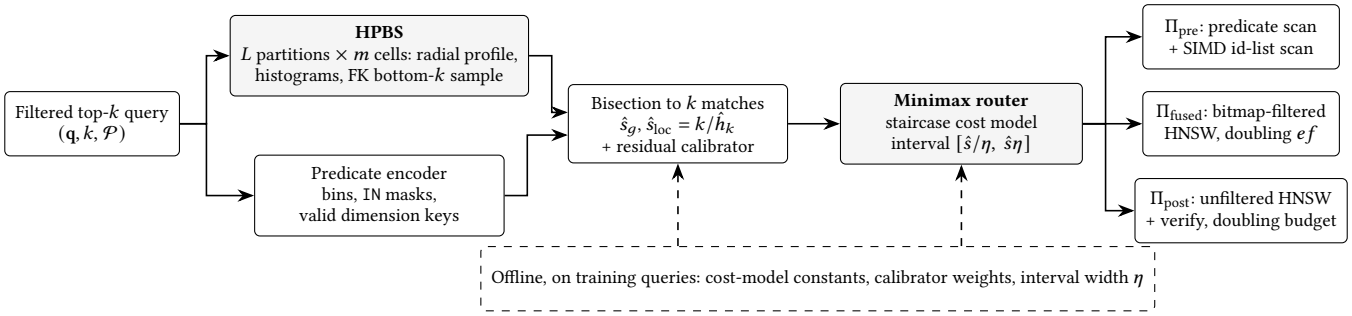


Figure 1: Architecture of HVO. Planning reads only the sketch, never the vector index. The router evaluates a calibrated cost model at both ends of the estimator's uncertainty interval and runs the plan with the smallest worst-case cost.

4.1 Structure

Partitions. We build L independent k-means partitions with m cells each, trained on a 2×10^5-vector sample with different seeds. For cell b we keep its centroid $\mathbf{c}_b$, its size N_b, and a *radial profile*: the 16 quantiles $\{\rho_{b,j}\}$ of the squared member-to-centroid distances $\|\mathbf{x} - \mathbf{c}_b\|^2$.

Attribute histograms. For each numeric attribute we fix $B = 32$ global equi-depth bins and store, per cell, the cumulative fraction of the cell's tuples below each bin edge (FP16). For integer-valued attributes (years, counts) the edges are moved to half-integers and an inclusive range $[l, u]$ is read as $[\lceil l \rceil - \frac{1}{2}, \lfloor u \rfloor + \frac{1}{2}]$. Without this, tied values make equi-depth edges collapse; a first run on real data produced P99 Q-errors above 10^4 for exactly this reason. For each categorical attribute we store per-cell value fractions. A range predicate is answered by linear interpolation inside its two boundary bins; an IN list sums the selected fractions.

Foreign-key samples. For each foreign key, a cell stores its $k_{\text{kmv}} = 64$ distinct keys with the smallest hash values together with the number of the cell's rows carrying each key. At query time the dimension predicate is evaluated on the (small) dimension table to obtain the valid key set $\mathcal{V}$. The cell's semi-join selectivity is estimated as

$$\hat{\sigma}_b^{\text{join}} = \frac{\sum_{h \in \mathcal{K}_b} f_b(h)\, \mathbb{I}[h \in \mathcal{V}]}{\sum_{h \in \mathcal{K}_b} f_b(h)}, \tag{3}$$

a ratio estimator over a uniform hash-sample of the cell's distinct keys that weights each key by its row count. Because $\mathcal{V}$ is materialized exactly, no threshold alignment between two bottom-k sketches is needed. The estimator is consistent but not unbiased, and its variance grows with key skew.

Cell predicate fraction. Within a cell we assume the constraints are independent: $\hat{\sigma}_b = \prod_j \hat{\sigma}_b^{(j)} \cdot \hat{\sigma}_b^{\text{join}}$. Correlation *across* cells, which is the spatial-relational correlation that defeats global estimators, is captured by the partition; only the within-cell residual is assumed away.

Why k-means and not random projections. Our original design used structured cross-polytope LSH cells ($2d_{\text{proj}}$ cones × 8 radial shells). Its cells follow random directions rather than the data, so a cell mixes unrelated neighborhoods. In our experiments it is the least accurate variant (P99 Q-error 31.59 against 9.83 for k-means cells at $L = 4$; Table 2).

Table 1: Algorithm 1: HPBS query-time estimation (one partition; the sketch returns the median over L).

Input: $\mathbf{q}$, k, predicate $\mathcal{P}$, partition $\{\mathbf{c}_b, N_b, \rho_{b,\cdot}, \text{hist}_b, \mathcal{K}_b\}_{b=1}^m$.
1: Encode $\mathcal{P}$: boundary bins and offsets of each range, bitmask of each IN list, valid key set $\mathcal{V}$ of the dimension predicate.
2: $\boldsymbol{\delta} \leftarrow \|\mathbf{c}\|^2 - 2\,\mathbf{C}\mathbf{q} + 1$ ▷ one sgemv
3: **for** each cell b: $\hat{\sigma}_b \leftarrow \prod_j \hat{\sigma}_b^{(j)} \cdot \hat{\sigma}_b^{\text{join}}$ (Eq. (3)); $\hat{s}_g$ += $N_b \hat{\sigma}_b / N$.
4: $C \leftarrow \{b : \hat{\sigma}_b > 0\}$; $lo \leftarrow 0$; $hi \leftarrow \max_{b \in C} \delta_b + \rho_{b,16}$.
5: **repeat** 26 times: $r \leftarrow (lo + hi)/2$; **if** $\sum_{b \in C} \hat{n}_b(r)\hat{\sigma}_b < k$ **then** $lo \leftarrow r$ **else** $hi \leftarrow r$; $C \leftarrow \{b \in C : \delta_b \le hi\}$.
6: **return** $\hat{s}_g$, $\hat{s}_{\text{loc}} \leftarrow \min(1, k/\sum_b \hat{n}_b(hi))$.

4.2 Estimation

Let $\delta_b = \|\mathbf{q} - \mathbf{c}_b\|^2$. For high-dimensional data the offset of a member from its centroid is nearly orthogonal to $\mathbf{q} - \mathbf{c}_b$, so $\|\mathbf{q} - \mathbf{x}\|^2 \approx \delta_b + \|\mathbf{x} - \mathbf{c}_b\|^2$. The estimated number of the cell's tuples inside the ball of squared radius r is therefore

$$\hat{n}_b(r) = \frac{N_b}{16} \left|\{j : \delta_b + \rho_{b,j} \le r\}\right|. \tag{4}$$

The estimated matches inside the ball are $\hat{M}(r) = \sum_b \hat{n}_b(r)\hat{\sigma}_b$. We find $\hat{r}$ with $\hat{M}(\hat{r}) = k$ by bisection (26 steps), and return

$$\hat{s}_{\text{loc}}^{(\ell)} = \min\Big(1,\ \frac{k}{\sum_b \hat{n}_b(\hat{r})}\Big), \qquad \hat{s}_g^{(\ell)} = \frac{1}{N} \sum_b N_b \hat{\sigma}_b. \tag{5}$$

The sketch returns the median over the L partitions. Algorithm 1 gives the procedure. Each partition costs one $m \times d$ matrix-vector product (Accelerate sgemv), m cell-fraction evaluations, and a bisection whose candidate set shrinks as the upper radius falls, giving $O(L(md + m|\mathcal{P}| + m\log(1/\epsilon)))$ time.

4.3 Residual calibrator

The raw estimate is biased where the orthogonality approximation in Eq. (4) is poor (the query lies close to a centroid) and where within-cell independence fails. A two-layer MLP $f_\Theta : \mathbb{R}^{12} \to \mathbb{R}$ with 16 ReLU units per layer (497 parameters) predicts the log-residual, and $\hat{s}_{\text{loc}} \leftarrow \text{clip}(\hat{s}_{\text{loc}}^{\text{raw}} e^{f_\Theta(\mathbf{z})}, k/N, 1)$. The twelve features are: $\ln \hat{s}_g$; $\ln \hat{s}_{\text{loc}}^{\text{raw}}$; the variance and the range of $\ln \hat{s}_{\text{loc}}^{(\ell)}$ across partitions; the number of predicate terms; a join indicator; the number of range terms; the number of IN terms; the log of the mean number of active cells; the mean and the standard deviation across partitions of the bisection radius $\hat{r}$; and $\ln \hat{s}_{\text{loc}}^{\text{raw}} - \ln \hat{s}_g$. It is trained on the

training split only, with squared log-error, L_2 penalty 10^{-3} and early stopping. Its forward pass takes about 4.1 μs in NumPy.

4.4 Error decomposition

Let $n_b(r)$ and $M_b(r)$ be the true numbers of tuples and of matches of cell b inside the ball of squared radius r, and $\sigma_b(r) = M_b(r)/n_b(r)$. Write $h(j)$ for the number of tuples within the distance of the j-th closest match, so $h_k = h(k)$.

Proposition 4.1 (Deterministic error decomposition). *Suppose that for every r and b the geometric and cell estimates satisfy*

$$\hat{n}_b(r) \in \big[n_b(r)/\alpha,\ \alpha n_b(r)\big], \qquad \hat{\sigma}_b \in \big[\sigma_b(r)/\beta,\ \beta\sigma_b(r)\big].$$

Then

$$Q\text{-}error(\hat{s}_{loc}, s_{loc}) \le \alpha \cdot \psi(\alpha\beta), \quad \psi(t) = \max\Big(\frac{h(\lceil tk\rceil)}{h(k)},\ \frac{h(k)}{h(\lfloor k/t\rfloor)}\Big).$$

If matches are locally uniform around $\mathbf{q}$ *($h(j) \propto j$), then $\psi(t) \approx t$ and the Q-error is at most $\alpha^2\beta$.*

Proof. With $M(r) = \sum_b n_b(r)\sigma_b(r)$,

$$\hat{M}(r) = \textstyle\sum_b \hat{n}_b\hat{\sigma}_b \in \big[M(r)/(\alpha\beta),\ \alpha\beta M(r)\big].$$

At the bisection point $\hat{M}(\hat{r}) = k$, so $M(\hat{r})$ lies in $[k/(\alpha\beta),\ \alpha\beta k]$. Since M and $n = \sum_b n_b$ are non-decreasing in r,

$$n(\hat{r}) \in \big[h(\lfloor k/(\alpha\beta)\rfloor),\ h(\lceil \alpha\beta k\rceil)\big].$$

Finally $\sum_b \hat{n}_b(\hat{r}) \in [n(\hat{r})/\alpha,\ \alpha n(\hat{r})]$, and $\hat{s}_{\mathrm{loc}}/s_{\mathrm{loc}} = h(k)/\sum_b \hat{n}_b(\hat{r})$. □

What the parameters control. Smaller cells (larger m) and a finer radial profile reduce α; larger B and k_{kmv} reduce the histogram and join parts of β. The remaining part of β is within-cell spatial-relational correlation, and ψ depends on how matches are arranged around $\mathbf{q}$. Neither is controlled by the sketch's parameters. We therefore make no distribution-free accuracy claim: Proposition 4.1 explains where error comes from, and Section 7.2 measures it. We prove nothing about the calibrator.

Space and maintenance. HPBS stores $Lm(d+16)$ floats for centroids and profiles, $Lm\sum_j B_j$ FP16 histogram entries, and Lmk_{kmv} key–count pairs, independent of N apart from the choice of m. With $L = 4$, $m = 1024$ it takes 5.1, 9.1 and 15.1 MiB for 128-, 384- and 768-d vectors (1.05%, 0.62% and 0.52% of the raw vectors), dominated by the centroids. It is built in 33–56 s, against 93–214 s for the HNSW index on 10 threads. Insertions update one cell per partition (counts, histogram entries, and a bottom-k insertion). Deletions can be handled by decrementing counts and dropping key entries, with lazy cell rebuilds. We have not evaluated maintenance under updates.

5 Plans and Hardware-Calibrated Cost Model

5.1 Plans

All plans are implemented on faiss [12] `IndexHNSWFlat` (inner product, $M = 32$, $ef_{\mathrm{construction}} = 200$) and run single-threaded.

- Π_{pre} evaluates $\mathcal{P}$ column-wise over all N rows (NumPy), takes the sorted id list, and runs a C top-k scan over the listed rows. The scan prefetches whole rows ahead and comes in three kernels: scalar FP32 (auto-vectorization disabled), NEON FP32 (4×4 FMA accumulators), and NEON INT8 (`SDOT` on symmetric int8 codes for the top $4k$, followed by an exact FP32 rerank).
- Π_{post} searches the unfiltered graph with budget e (search queue and result list of size e), verifies $\mathcal{P}$ on the candidates, and doubles e from 64 until k matches are found, up to 2^{17}. Restarting rounds instead of resuming costs at most a factor of about two over a resumable implementation.
- Π_{fused} materializes $\mathcal{P}$ as a bitmap and passes it to faiss's HNSW search as an `IDSelectorBitmap`, so non-matching nodes are traversed but never returned. It doubles ef from 64 until k results are found *and* a doubled budget returns the same set. We added the second condition after observing recall of 0.23 at $s_{\mathrm{loc}} < 1\%$ when stopping at the first k results.

Because the plans adapt their budgets, a plan's latency does not depend on the optimizer's estimate; the optimizer only chooses which plan runs. This makes the oracle well defined.

5.2 Cost model

Staircase plans. A graph plan whose rounds have budgets $e_i = e_0 2^i$ and which stops once the budget reaches a need ν runs $j(\nu) = \lceil \log_2(\nu/e_0)\rceil^+$ rounds, capped at the budget limit. If a round of budget e costs $b_0 + b_1 e^\gamma$, the plan costs

$$\mathrm{St}(\nu; b_0, b_1, \gamma) = \sum_{i=0}^{j(\nu)} \big(b_0 + b_1 (e_0 2^i)^\gamma\big). \tag{6}$$

Plan costs. With $C_{\mathrm{rel}} = c_0 + c_1 N|\mathcal{P}| + c_2 N\,\mathbb{I}[\mathrm{join}]$ for columnar predicate evaluation,

$$C(\Pi_{\mathrm{pre}}) = C_{\mathrm{rel}} + a_0 + c_{\mathrm{scan}}\, s_g N, \tag{7}$$

$$C(\Pi_{\mathrm{post}}) = \mathrm{St}\big(k/s_{\mathrm{loc}};\ \theta_{\mathrm{post}}\big), \tag{8}$$

$$C(\Pi_{\mathrm{fused}}) = C_{\mathrm{rel}} + \mathrm{St}^{+1}\big(c_f (k/s_{\mathrm{loc}})^{\beta_f};\ \theta_{\mathrm{fused}}\big), \tag{9}$$

where St^{+1} adds the confirmation round. The need of Π_{post} is exactly $h_k = k/s_{\mathrm{loc}}$ (Definition 3.1). Filtered traversal needs a budget that grows faster than h_k, which we capture with β_f.

Calibration. The constants are fitted on the training queries using their *true* selectivities and measured latencies: (c_0, c_1, c_2) and (a_0, c_{scan}) by non-negative least squares on relative error, and θ by nonlinear least squares in log space ($\gamma \in [0.3, 2.5]$, with c_f and β_f chosen on a grid). On SYN-128 the fit gives $c_1 = 0.53$ ns per row and predicate term and $c_{\mathrm{scan}} = 16.0$ ns per vector for the NEON kernel. It also gives $\gamma_{\mathrm{post}} = 1.70$, $\gamma_{\mathrm{fused}} = 2.42$, and $\beta_f = 1.25$. The super-linear γ is a property of faiss's HNSW search: its candidate queue makes large budgets disproportionately expensive. One round at $e = 65{,}536$ cost about 2 s on SYN-128. A first model that assumed $\gamma \le 1$, as the analysis of the original draft did, underpredicted the cost of fused plans by up to 10× and misrouted tail queries even when given the true selectivities.

Hardware-agnostic ablation. To isolate the hardware calibration we also fit the same staircases to *distance-evaluation counts* instead of time, so that a sequential and a random distance computation cost the same, as in cost models that count distance evaluations.

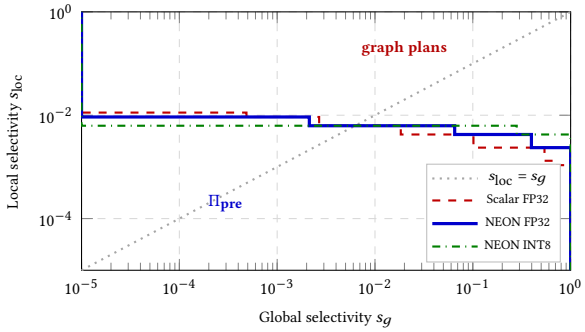


Figure 2: Pre-filter boundary $s_g^*(s_{\text{loc}})$ of Theorem 5.1 on SYN-128 for the three scan kernels (cost model calibrated on the training split; two predicate terms, no join). Below and to the left of each curve pre-filtering is cheapest. Faster kernels push the boundary to larger s_g. Above $s_{\text{loc}} \approx 1\%$ a graph plan wins for every s_g, because columnar predicate evaluation already costs $\Theta(N)$.

5.3 Phase boundary

Theorem 5.1 (Pre-filter boundary). *Let $C_G(s_{loc})$ denote the cost of the cheaper graph plan, $\min(C(\Pi_{fused}), C(\Pi_{post}))$. Under the cost model of Eqs. (7) to (9), the pre-filter plan is optimal if and only if*

$$s_g \le s_g^*(s_{loc}) = \frac{C_G(s_{loc}) - C_{rel} - a_0}{c_{scan} N}.$$

If $C_G(s_{loc}) < C_{rel} + a_0$, Π_{pre} is never optimal. Holding the other terms fixed, replacing a scan kernel by one that is λ times faster multiplies s_g^* by λ.

Proof. $C(\Pi_{\text{pre}})$ is affine and increasing in s_g, and C_G does not depend on s_g; equate and solve. The scaling follows because c_{scan} enters only in the denominator. □

Theorem 5.1 has a consequence that matters in practice. When predicates are evaluated by scanning columns, C_{rel} is linear in N, so pre-filtering pays $\Theta(N)$ even for very selective predicates. For local selectivities above roughly 1–2%, a graph plan is then cheaper whatever s_g is. The boundary becomes nearly a horizontal line in the (s_g, s_{loc}) plane, and global selectivity carries little information about the right plan (Figure 2). The kernel enters through c_{scan}, which on SYN-128 is 64.4, 16.0, and 3.3 ns per vector for the scalar, NEON FP32 and INT8 kernels.

5.4 Routing

Point routing picks $\arg\min_\Pi C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}})$. *Minimax routing* uses an interval $I = [\hat{s}_{\text{loc}}/\eta,\ \hat{s}_{\text{loc}}\eta]$ and picks

$$\begin{aligned}\hat{\Pi} &= \arg\min_{\Pi} \max_{s \in I} C(\Pi; \hat{s}_g, s) \\ &= \arg\min_{\Pi} \max\left(C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}/\eta),\ C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}\eta)\right),\end{aligned} \tag{10}$$

since every plan cost is monotone in s_{loc}. We set η to the 0.9-quantile of the estimator's Q-error on the training split, fixed before looking at test data ($\eta = 1.86$ and 2.06 for HPBS). Because $C(\Pi_{\text{pre}})$ does not depend on s_{loc}, minimax routing prefers the plan whose cost is certain whenever the estimate is uncertain.

Runtime guard. We also evaluated an early-exit rule that aborts Π_{post} after its first round if the observed match rate is below half the estimate and the cost model predicts Π_{pre} to be cheaper than the remaining rounds. It is replayed exactly from the recorded per-round timings.

6 Regret Analysis

Regret is $\mathcal{R} = C(\hat{\Pi}; s)/C(\Pi^*; s)$, where Π^* minimizes cost at the true selectivities $s = (s_g, s_{\text{loc}})$. The results below concern the cost model of Section 5. Measured regret differs from model regret by the model's calibration error, which we report in Section 7. All statements hold per query, given that query's estimation error.

Lemma 6.1 (Sensitivity of staircase plans). *Let a plan's cost be $A + \mathrm{St}(c\,(k/s)^\beta; b_0, b_1, \gamma)$ with $A, b_0, b_1 \ge 0$ and $\gamma > 0$. If $\hat{s}/s \in [1/\eta, \eta]$, then its cost at $\hat{s}$ is within a factor $\Lambda_{\beta,\gamma}(\eta)$ of its cost at s, where*

$$\Lambda_{\beta,\gamma}(\eta) = 1 + \delta\, 2^{\gamma\delta}, \qquad \delta = \lceil \beta \log_2 \eta \rceil.$$

Proof. The needs differ by a factor of at most η^β, so the round counts differ by at most δ, because $\lceil x + y \rceil \le \lceil x \rceil + \lceil y \rceil$. Suppose the larger cost has $j + \delta$ rounds and the smaller j (fewer additional rounds only shrink the ratio). The constant parts have ratio $(j + \delta + 1)/(j + 1) \le 1 + \delta$. With $r = 2^\gamma$ the geometric parts have ratio

$$\frac{\sum_{i \le j+\delta} r^i}{\sum_{i \le j} r^i} = 1 + \frac{r^{j+1}}{\sum_{i \le j} r^i} \sum_{i < \delta} r^i \le 1 + r \sum_{i < \delta} r^i \le 1 + \delta r^\delta,$$

using $\sum_{i \le j} r^i \ge r^j$ and $r \ge 1$. A ratio of sums is at most the largest ratio of its parts, and adding $A \ge 0$ to both costs only decreases the ratio. Capping the round count at the budget limit also only decreases it. □

The factor is 1 when $\eta < 2^{1/\beta}$ *and* no doubling boundary lies between the two needs. In the worst case, though, an arbitrarily small error can cross a boundary, which costs one extra round and gives the factor $1 + 2^\gamma$. This discreteness is a real property of adaptive plans, not an artifact of the proof.

Let $\Lambda = \max(\eta_g, \Lambda_{1,\gamma_{\text{post}}}(\eta), \Lambda_{\beta_f,\gamma_{\text{fused}}}(\eta))$, where η_g bounds the multiplicative error of $\hat{s}_g$, which enters $C(\Pi_{\text{pre}})$ affinely.

Theorem 6.2 (Point routing). *If $\hat{s}_g/s_g \in [1/\eta_g, \eta_g]$ and also $\hat{s}_{loc}/s_{loc} \in [1/\eta, \eta]$, then point routing has $\mathcal{R} \le \Lambda^2$. If, in addition, $\hat{s}_g = s_g$ and either $\hat{\Pi}$ or Π^* is Π_{pre}, then $\mathcal{R} \le \Lambda$.*

Proof. By Lemma 6.1, $C(\Pi; \hat{s}) \in [C(\Pi; s)/\Lambda,\ \Lambda C(\Pi; s)]$ for every plan. Hence $C(\hat{\Pi}; s) \le \Lambda C(\hat{\Pi}; \hat{s}) \le \Lambda C(\Pi^*; \hat{s}) \le \Lambda^2 C(\Pi^*; s)$. If $\hat{\Pi} = \Pi_{\text{pre}}$ and its cost is exact, the first factor is 1. If $\Pi^* = \Pi_{\text{pre}}$, the last factor is 1. □

Theorem 6.3 (Minimax routing). *Suppose $\hat{s}_g = s_g$ and $s_{loc} \in I = [\hat{s}_{loc}/\eta, \hat{s}_{loc}\eta]$. Then minimax routing (Eq. (10)) satisfies: (i) if $\Pi^* = \Pi_{pre}$, then $\mathcal{R} = 1$; (ii) in all cases $\mathcal{R} \le \Lambda(\eta^2)$.*

Proof. Let $\bar{C}(\Pi) = \max_{s' \in I} C(\Pi; s')$; since $s_{\text{loc}} \in I$, $C(\Pi; s) \le \bar{C}(\Pi)$. (i) $C(\Pi_{\text{pre}})$ does not depend on s_{loc}, so $\bar{C}(\Pi_{\text{pre}}) = C(\Pi_{\text{pre}}; s)$, and $C(\hat{\Pi}; s) \le \bar{C}(\hat{\Pi}) \le \bar{C}(\Pi_{\text{pre}}) = C(\Pi^*; s)$. (ii) $C(\hat{\Pi}; s) \le \bar{C}(\hat{\Pi}) \le \bar{C}(\Pi^*) \le \Lambda(\eta^2) C(\Pi^*; s)$, because any $s' \in I$ is within a factor η^2 of s_{loc}. □

Theorem 6.3 is the formal reason for minimax routing. Misrouting a query whose best plan is the exact pre-filter scan to a graph plan is the failure that produces the worst tail latencies: the graph plan can be orders of magnitude slower. Minimax routing provably avoids it whenever the interval covers the truth. The price is conservatism: when the interval is wide, the router may scan where a graph plan would have been faster.

Proposition 6.4 (Global-selectivity routing). *Let a router choose its plan as any function of s_g alone. Let graph-plan costs satisfy $C_G(1) \le C_1$ and $C_G(k/N) \ge bN^\gamma$ for constants $C_1, b, \gamma > 0$, and let $C(\Pi_{pre}) = a + c's_gN$, which holds when predicates are evaluated through an index at a cost proportional to their output. Then for N large enough that $k \le \sqrt{bN^\gamma C_1/c'}$, its worst-case regret over instances with that N is at least $\sqrt{bN^\gamma/(c'C_1)} \,/\, (1 + o(1))$, that is, $\Omega(N^{\gamma/2})$.*

Proof. Fix $s^* = \sqrt{bN^\gamma C_1/c'}/N \ge k/N$. Matching tuples can be placed around $\mathbf{q}$ ($s_{\text{loc}} = 1$) or beyond all non-matching tuples ($s_{\text{loc}} = k/N$) while keeping $s_g = s^*$, and the router cannot tell the two instances apart. If it picks Π_{pre}, the first instance gives $\mathcal{R} \ge c's^*N/C_1$. If it picks a graph plan, the second gives $\mathcal{R} \ge bN^\gamma/(a + c's^*N)$. Both equal $\sqrt{bN^\gamma/(c'C_1)}$ up to the lower-order term a. □

With columnar predicate evaluation, $C(\Pi_{\text{pre}}) \ge c_1N$ for every s_g. The same argument then gives regret $\Omega(\min(N, N^{\gamma-1}))$. This is a constant set by the ratio of random-access to per-row predicate cost when $\gamma \le 1$, and it still grows with N for the superlinear γ we measure. HVO's bounds, by contrast, depend only on the estimation error. This contrast is why we measure the estimator's tail and not only its median.

7 Experimental Evaluation

We ask four questions. How accurate and how fast is HPBS, compared with histogram, exact-count, sampling and index-probing estimators (Section 7.2)? Does better estimation translate into lower end-to-end latency (Section 7.3)? How do the scan kernel and each component matter (Section 7.4)? And how does measured regret compare with the bounds (Section 7.5)?

7.1 Setup

Hardware and software. Apple M4 (4 performance and 6 efficiency cores, 16 GB unified memory), macOS, Python 3.14, NumPy (Accelerate BLAS), faiss-cpu 1.13.2, Apple clang with `-O3 -mcpu=native`. All latencies are measured on one thread (OpenMP and Accelerate both limited to one thread). Each plan is run once as a warm-up and then twice; we report the slower of the two timed runs. A first run of the 768-d benchmark was invalidated because two copies of the vectors (table and index) exceeded memory and the machine swapped. In the reported runs the table aliases the index's vector storage. We logged swap activity every minute: the remaining swap-ins came from other processes paging in, and the timed process stayed CPU-bound (above 85% CPU utilization whenever we checked).

Synthetic data. Each synthetic benchmark has $N = 10^6$ unit-norm vectors from a two-level mixture (64 topics × 16 subtopics with Zipfian popularity) in $d = 128$ (SYN-128) or $d = 768$ (SYN-768). Attributes are generated from the latent topic with graded correlation. `category` (32 values) equals the topic's value with probability 0.7; `lang` (12 values) follows the subtopic with probability 0.5; `year` has a topic-specific mean plus Gaussian noise; `rating` has a weak subtopic effect; `price` (log-normal) and `views` (Pareto) are independent of the vectors. A foreign key references a 20,000-row seller table with `region` (10 values) and `tier` (4 values). For 60% of rows the seller is drawn, with Zipfian popularity, from the topic's preferred region.

Real data (ARXIV-384). We sample 10^6 papers, plus a disjoint pool of 3000 query papers, from the CC0-licensed arXiv-abstracts-2021 dump (about 2×10^6 papers) [30]. Each paper is embedded with the sentence encoder all-MiniLM-L6-v2 [31] ($d = 384$, title and abstract truncated to the model's 128-token training length, unit norm). All attributes are real metadata:

- `archive`: the top-level archive of the primary category (31 archives plus "other");
- `published`: whether the paper has a journal reference;
- `year`: the submission year, taken from the identifier (1991–2021);
- `n_cats`: the number of listed categories;
- `n_authors` and `abs_words`: the author count and abstract length.

A foreign key references a dimension table of the 172 primary categories, with their subject `group` (8 values: physics, mathematics, computer science, …) and a popularity quartile. Spatial-relational correlation is not injected here: it arises because a paper's text determines both its embedding and its field.

Queries. On synthetic data a query vector is a data vector plus Gaussian noise of norm about 0.35, renormalized; on ARXIV-384 it is the embedding of a held-out paper. Its predicate is a conjunction of 1–3 terms drawn independently of the vector: numeric ranges of random width 2–90% of the quantile range, `IN` lists, and semi-joins on the two dimension attributes. Because predicates are drawn independently of the query, correlation arises naturally: a `category` or `archive` filter either includes or excludes the query's topic or field. We generate 1000 training and 1500 test queries per benchmark from disjoint seeds and drop queries with no match. The test splits contain 640/605/255 (SYN-128), 647/605/248 (SYN-768) and 630/427/413 (ARXIV-384) anti-correlated/independent/positively correlated queries. They include 56, 60 and 164 queries with $s_{\text{loc}} < 10^{-3}$; real data has far more of them. Ground truth (exact s_g, h_k, and filtered top-10) comes from brute force. Everything fitted—the calibrator, the cost model, the thresholds of the global routers, and the interval width η—uses only the training split.

Baselines. The estimators are:

- *1-D histograms + AVI* (PostgreSQL-style): 100 equi-depth bins per continuous column and exact value frequencies for categorical and integer-valued columns (as most-common-value lists would give), independence across terms, a uniform foreign-key assumption, and $\hat{s}_{\text{loc}} = \hat{s}_g$.
- *Exact s_g + AVI* (Milvus-style): the exact predicate count, with $\hat{s}_{\text{loc}} = s_g$.
- *1% uniform sample*: the rank of the first sampled match among sampled tuples.

- *HNSW probe* (Exqutor-style): the match rate among the first 256 unfiltered neighbors returned by the index ($ef = 256$).

Each is fed to the same cost model, with point and with minimax routing. The routers additionally include the three static plans and global-selectivity routers with two-way ($\Pi_{\text{pre}}/\Pi_{\text{post}}$) and three-way thresholds on the exact s_g, tuned on the training split to minimize mean latency (which is generous to them). We do not run `pgvector`, Milvus, DuckDB or ACORN themselves: these policies isolate the routing decision on identical plan implementations.

Metrics. The oracle runs all three plans and picks the fastest with Recall@10$\geq$ 0.9; the other routers are charged their plan's latency plus their planning time. Measured regret is the ratio of a router's latency to the oracle's. Plan accuracy is agreement with the oracle's choice, which penalizes near-ties, so we emphasize latency and regret.

7.2 Selectivity estimation

Table 2 and Figure 3 report Q-error on the test split.

Independence-based estimators fail under correlation. Both histogram-based AVI and exact-count AVI reach P99 Q-errors of 25.95 and 26.21 on SYN-128. Their errors are concentrated in the anti-correlated regime and grow sharply for rare local matches (165.49 and 169.67). Knowing s_g exactly does not help, because the error is in assuming $s_{\text{loc}} = s_g$. The exact count also costs 1512 μs, because it evaluates the predicate over all rows during planning.

Sampling starves. The 1% sample (P99 24.45) bases its estimate on very few sampled matches near the query and is noisy across all regimes.

Index probing is accurate on average but fails on rare matches. The HNSW probe is the strongest baseline (P99 5.67 and 6.06). When no match appears among the first 256 neighbors it can only report an upper bound, so its rare-match P99 is 25.53 and 12.97. It also costs 110 and 163 μs of random index accesses.

Real data is harder. On ARXIV-384 the tails are an order of magnitude heavier for every estimator. Text determines both a paper's embedding and its field, so correlation is strong: exact-count AVI reaches a P99 Q-error of 514.26 and histograms 765.57. The HNSW probe's P99 rises to 39.23 and 134.39 on rare matches. The calibrated sketch reaches 6.17 (15.59 on rare matches), the best of all estimators, and the 1% sample is second (18.41), at a memory footprint of about 15 MiB and 258 μs per estimate.

HPBS. The calibrated sketch has the lowest P99 Q-error on every benchmark (3.07, 4.72 and 6.17), and it stays the most accurate on rare local matches (2.25, 1.91 and 15.59). That is the regime where index probing and histograms are least accurate and where misrouting is most expensive. The raw sketch is already accurate at the median and on rare matches. The calibrator mainly removes its tail, reducing the P99 from 9.83 to 3.07 on SYN-128 and from 9.95 to 6.17 on ARXIV-384. One partition ($L = 1$) is almost as accurate after calibration (4.25 and 6.20) at 50% and 35% of the planning time. HPBS is less accurate than the probe at the median for well-populated neighborhoods; its advantage lies in the tail.

7.3 End-to-end routing

Table 3 reports latency, recall and regret for every router, and Figure 3(c) shows where the plans cross.

No static plan is acceptable. Pre-filtering always pays the full predicate scan (mean 4.53 ms on SYN-128). Post-filtering is the best plan for most queries, but its P99 is 548.34 ms because anti-correlated queries force it through a large part of the graph. Static fused traversal is worse still, because filtered traversal without two-hop expansion loses connectivity at low s_{loc}.

Global-selectivity routing has a large regret tail. Tuned two-way thresholds give a mean of 1.48 ms and 2.87 ms with P99 regret 43.58 and 43.97. Every query above the threshold goes to post-filtering regardless of s_{loc}, as Proposition 6.4 predicts. The three-way variant finds no profitable fused band on SYN-128, where its thresholds collapse to the two-way rule. On SYN-768 it lowers the mean to 2.64 ms. Its regret tail stays large.

Correlation-blind estimates poison a good cost model. Feeding the same cost model with histogram or exact-count AVI estimates is worse than the simple threshold (mean 15.48 and 15.92 ms): the model trusts $\hat{s}_{\text{loc}} = s_g$ and sends anti-correlated queries to graph plans.

HVO. With HPBS estimates and minimax routing, HVO has a mean latency of 0.96 ms and 2.04 ms. That is 35% and 23% below the best tuned global router (1.48 and 2.64 ms) and close to the oracle (0.79 and 1.83 ms). On SYN-768 it matches the cost model fed the *true* selectivities (2.06 ms), so estimation is no longer the bottleneck there. Its P99 regret is 6.14 and 6.08, against 43.58 and 43.97 for the two-way router. On the synthetic benchmarks HVO does *not* improve P99 latency over the two-way threshold rule (3.97 vs. 3.68 ms on SYN-128, 14.63 vs. 9.06 ms on SYN-768). That rule sends many queries to the pre-filter scan, whose cost is bounded, so its slowest queries are no slower than a scan; it pays for this in the mean. HVO's tail consists of graph plans that the cost model priced correctly on average but that run long for individual queries. Point routing with the same estimates has a similar mean but a much heavier latency tail (12.05 ms P99 on SYN-128). That is the failure Theorem 6.3 rules out. The probe-based cost model is the strongest competitor. With minimax routing it comes within a few percent of HVO (1.03 vs. 0.96 ms), because both estimators are accurate where most queries lie. HVO's advantage is that it never touches the vector index during planning and that it remains accurate on rare local matches.

Real data. On ARXIV-384 the gap between routers widens, because local selectivity there often differs from global selectivity by orders of magnitude:

- *Correlation-blind estimates are catastrophic.* The cost model fed exact-count AVI estimates averages 211.83 ms, because it sends anti-correlated queries into graph plans that traverse most of the graph.
- *Global thresholds are a poor fit.* The best tuned global router averages 5.25 ms.
- *HVO.* It averages 1.84 ms, close to the cost model fed true selectivities (1.77 ms) and to the oracle (1.53 ms). Unlike on the synthetic benchmarks, its P99 latency (13.95 ms) is also below the threshold rule's (19.04 ms) and close to the oracle's (13.67 ms); its P99 regret is 5.52, against 108.75.
- *Index probing.* The probe-based cost model is again the closest competitor (1.98 ms, 2.21 ms with minimax routing).

Table 2: Local-selectivity Q-error on the test split, by correlation regime and for rare local matches ($s_{\text{loc}} < 10^{-3}$), and median planning-time cost of the estimator. Lower is better.

	Anti-correlated			Independent			Positively correlated			Rare	Time
Estimator	P50	P95	P99	P50	P95	P99	P50	P95	P99	P99	(μs)
SYN-128											
1-D hist. + AVI (PG-style)	3.00	12.16	67.32	1.24	1.74	1.89	2.19	8.15	13.45	165.49	9
Exact s_g, s_{loc}=s_g (Milvus-style)	2.97	14.02	73.38	1.23	1.68	1.78	2.20	8.19	11.13	169.67	1512
Uniform sample 1%	2.22	11.20	28.69	1.90	9.10	18.98	1.59	6.40	14.14	133.81	80
HNSW probe, P=256 (Exqutor-style)	1.27	3.69	6.73	1.16	2.17	3.24	1.16	1.71	2.56	25.53	110
HPBS, cross-polytope cells (m=256)	1.79	10.96	27.67	3.37	18.81	23.98	14.31	30.87	36.26	59.82	124
HPBS, k-means m=256	1.48	8.19	16.66	3.96	24.52	38.12	13.92	44.49	57.31	1.90	58
HPBS, L=1	1.24	2.52	5.60	1.71	6.69	10.04	3.42	13.00	18.70	1.92	38
HPBS, L=1 + calibrator	1.31	2.44	3.64	1.25	2.12	3.04	1.54	3.93	6.89	2.45	42
HPBS (L=4, m=1024), raw	1.24	2.40	5.32	1.69	5.70	8.59	3.35	9.75	13.54	1.76	81
HPBS + calibrator	1.29	2.14	2.96	1.27	2.00	2.75	1.42	2.67	4.07	2.25	85
SYN-768											
1-D hist. + AVI (PG-style)	2.97	14.35	58.44	1.24	1.75	2.05	2.30	7.20	18.73	136.50	10
Exact s_g, s_{loc}=s_g (Milvus-style)	2.96	16.38	58.94	1.23	1.72	1.80	2.30	6.83	19.01	139.95	1516
Uniform sample 1%	2.22	11.33	32.87	1.82	9.34	20.42	1.60	6.55	15.31	10.88	530
HNSW probe, P=256 (Exqutor-style)	1.27	4.67	7.71	1.16	1.92	3.76	1.17	1.88	2.27	12.97	163
HPBS, cross-polytope cells (m=256)	1.91	10.79	22.05	3.42	18.91	28.27	12.04	29.96	35.77	58.01	134
HPBS, k-means m=256	1.52	9.16	19.17	4.43	32.46	49.34	15.31	54.67	78.57	1.94	62
HPBS, L=1	1.26	3.16	10.37	1.77	10.66	20.22	3.41	16.26	23.05	1.62	47
HPBS, L=1 + calibrator	1.36	2.62	4.74	1.27	2.04	2.62	1.66	5.32	8.45	2.24	51
HPBS (L=4, m=1024), raw	1.25	2.78	6.90	1.82	8.57	13.57	3.87	13.76	18.60	1.86	143
HPBS + calibrator	1.33	2.34	3.68	1.25	2.00	2.61	1.60	4.46	6.84	1.91	147
ARXIV-384 (real)											
1-D hist. + AVI (PG-style)	7.49	459.83	1265.85	1.23	3.42	17.65	1.67	7.31	24.04	2002.20	3
Exact s_g, s_{loc}=s_g (Milvus-style)	6.23	238.56	1133.30	1.21	1.63	1.76	1.71	5.43	18.67	1312.87	1512
Uniform sample 1%	2.80	10.32	30.67	1.80	6.27	11.58	1.30	3.82	7.97	53.68	258
HNSW probe, P=256 (Exqutor-style)	1.46	17.95	45.71	1.16	1.89	18.39	1.12	1.75	2.51	134.39	276
HPBS, cross-polytope cells (m=256)	5.26	55.64	151.79	6.77	21.34	26.40	21.23	31.46	36.74	287.10	137
HPBS, k-means m=256	1.80	12.45	26.78	6.29	20.55	27.42	18.14	32.31	41.52	27.32	78
HPBS, L=1	1.47	4.80	8.35	1.99	5.36	7.25	4.48	9.13	11.93	24.69	53
HPBS, L=1 + calibrator	1.54	4.02	7.80	1.24	2.16	2.78	1.51	3.42	5.36	13.87	58
HPBS (L=4, m=1024), raw	1.44	4.63	8.07	1.97	5.23	6.41	4.67	8.76	11.02	23.29	144
HPBS + calibrator	1.52	4.02	7.03	1.28	2.23	3.39	1.53	2.95	6.19	15.59	148

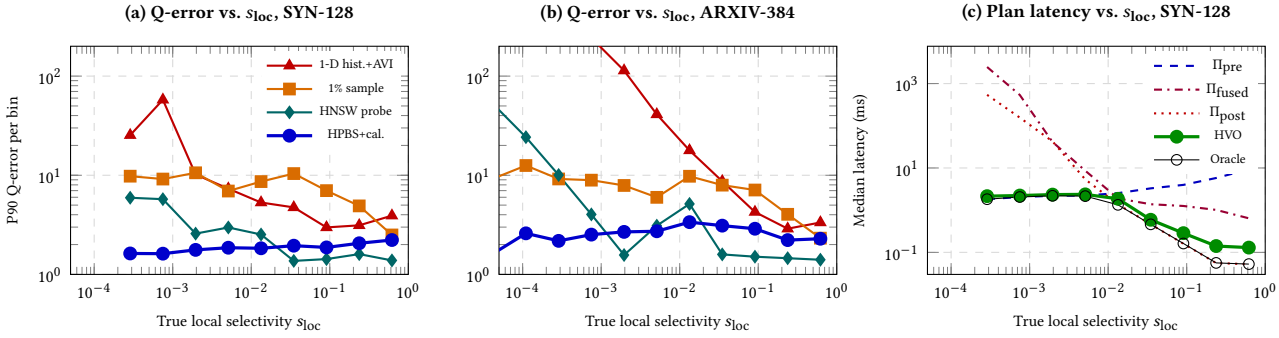


Figure 3: (a, b) P90 Q-error within bins of true local selectivity: the baselines degrade as matches become rare near the query, and HPBS does not. (c) Median latency of each plan by true local selectivity on SYN-128, with HVO (including planning time) and the oracle. Pre-filtering is flat in s_{loc}; the graph plans' cost explodes as s_{loc} falls.

Its tail estimation errors cost it more here than on synthetic data.

- *One partition.* With $L = 1$, HVO averages 1.66 ms: at real-data latencies, the planning time saved outweighs the small loss in accuracy.

What limits HVO. Three factors account for the rest of the gap to the oracle:

- *Planning time.* Excluding it, HVO's choices reach 0.82 and 1.85 ms. At 85 and 147 μs, planning matters when the best plan itself takes a few hundred microseconds. Roughly a third of it is Python orchestration around the C kernel.
- *Cost-model calibration.* For the best plan the model's error is median $\kappa_{\text{cal}} = 1.20$ (P95 1.69) on SYN-128 and 1.36 (P95 3.13) on SYN-768.
- *Recall.* The oracle only accepts plans with Recall@10 ≥ 0.9, while the routers' graph plans occasionally return fewer

Table 3: End-to-end latency (ms, including planning time), recall, plan accuracy against the oracle, measured regret, and median planning time (μs) on the test split. CM: the calibrated cost model of Section 5. "Robust" denotes minimax routing with the interval width fitted on the training split.

Router	Mean	P50	P95	P99	Recall@10	Plan acc. (%)	Mean regret	P99 regret	Plan (μs)
SYN-128									
Always pre-filter	4.53	3.66	10.31	10.82	1.000	18.7	50.21	260.33	0
Always post-filter	33.74	0.22	48.18	548.34	0.978	74.6	17.46	272.28	0
Always fused	117.87	1.35	46.16	2605.96	0.983	6.7	67.54	1460.15	0
Global s_g threshold (2-way)	1.48	0.29	3.07	3.68	0.982	73.4	2.62	43.58	0
Global s_g thresholds (3-way)	1.48	0.29	3.07	3.68	0.982	73.4	2.62	43.58	0
CM + 1-D histograms	15.48	0.23	12.25	147.31	0.978	80.4	8.21	51.45	9
CM + exact s_g, AVI	15.92	1.74	14.30	153.32	0.978	80.5	16.95	63.98	1512
CM + 1% sample	6.20	0.31	4.84	43.56	0.978	79.7	3.58	22.26	80
CM + HNSW probe	1.09	0.34	3.44	12.52	0.978	86.6	1.99	5.15	110
CM + HNSW probe, robust	1.03	0.34	2.98	4.20	0.979	81.1	2.02	4.09	110
HVO, point routing	1.06	0.38	3.15	12.05	0.978	86.9	1.99	5.89	85
HVO (robust)	0.96	0.39	2.99	3.97	0.979	83.9	2.03	6.14	85
CM + true selectivities	0.84	0.22	2.72	7.98	0.978	89.3	1.04	3.95	0
Oracle	0.79	0.26	2.53	3.87	0.993	100.0	1.00	1.00	0
SYN-768									
Always pre-filter	20.25	12.07	65.81	77.77	1.000	19.9	102.49	776.11	0
Always post-filter	41.94	0.42	62.74	577.76	0.959	69.3	18.67	259.81	0
Always fused	160.87	1.61	60.35	2702.76	0.974	10.8	71.50	1521.64	0
Global s_g threshold (2-way)	2.87	0.64	8.11	9.06	0.965	70.1	3.01	43.97	0
Global s_g thresholds (3-way)	2.64	0.64	6.48	15.42	0.964	70.5	2.29	25.34	0
CM + 1-D histograms	8.37	0.45	15.24	166.55	0.958	77.1	2.85	40.75	10
CM + exact s_g, AVI	16.52	1.97	17.44	168.89	0.958	76.8	8.69	41.15	1516
CM + 1% sample	9.11	0.99	10.60	53.39	0.958	76.1	4.28	14.45	530
CM + HNSW probe	2.21	0.80	7.32	17.50	0.959	80.5	1.68	4.76	163
CM + HNSW probe, robust	2.15	1.02	7.60	16.72	0.959	69.9	1.86	6.64	163
HVO, point routing	2.15	0.68	6.45	18.00	0.957	82.2	1.68	4.76	147
HVO (robust)	2.04	0.75	6.70	14.63	0.959	74.9	1.77	6.08	147
CM + true selectivities	2.06	0.42	6.64	18.15	0.957	82.4	1.12	4.33	0
Oracle	1.83	0.63	5.87	13.87	0.989	100.0	1.00	1.00	0
ARXIV-384 (real)									
Always pre-filter	14.94	11.20	36.57	41.30	1.000	25.0	108.49	423.79	0
Always post-filter	691.43	0.17	3484.74	15122.73	0.968	70.5	308.50	8686.66	0
Always fused	833.57	1.47	8486.38	15509.94	0.978	4.5	365.75	8961.50	0
Global s_g threshold (2-way)	5.25	1.53	9.95	19.04	0.992	71.7	9.86	108.75	0
Global s_g thresholds (3-way)	14.74	1.82	10.03	18.39	0.994	54.6	12.21	109.52	0
CM + 1-D histograms	309.84	0.18	384.78	13921.74	0.980	77.6	92.26	2357.76	3
CM + exact s_g, AVI	211.83	1.71	212.66	12883.01	0.982	80.7	45.66	1401.86	1512
CM + 1% sample	15.62	0.47	10.10	68.53	0.984	87.6	4.70	16.71	258
CM + HNSW probe	1.98	0.60	7.94	15.85	0.987	92.4	2.68	7.14	276
CM + HNSW probe, robust	2.21	0.70	8.76	15.85	0.989	79.0	3.19	11.84	276
HVO, point routing	2.01	0.35	7.66	15.66	0.984	91.0	1.84	4.67	148
HVO (robust)	1.84	0.35	7.59	13.95	0.988	88.6	1.91	5.52	148
CM + true selectivities	1.77	0.17	7.29	14.91	0.984	92.7	1.04	2.48	0
Oracle	1.53	0.18	7.07	13.67	0.992	100.0	1.00	1.00	0

true neighbors. Their mean recall is 0.979 and 0.959 against 0.993 and 0.989 for the oracle. A recall-aware cost model is future work.

7.4 Hardware kernels and ablations

Scan kernels. Table 4 repeats routing with each pre-filter kernel. The NEON FP32 kernel is 4.0× (SYN-128) and 5.2× (SYN-768) cheaper per vector than the scalar kernel, and INT8 `SDOT` with FP32 rerank is another 4.9× and 4.3× cheaper. Faster kernels widen the pre-filter region (Figure 2), and the oracle picks Π_{pre} more often. The calibrated cost model follows the shift, because only c_{scan} changes.

Component ablations. Table 5 removes one component at a time:

- *Hardware-agnostic cost.* Replacing time-calibrated costs by distance-evaluation counts makes sequential and random distance computations cost the same. HVO's mean rises from 0.96 to 1.92 ms on SYN-128 and from 1.84 to 4.78 ms on ARXIV-384.

Table 4: Scan kernel sensitivity. c_{scan}: fitted cost per scanned vector. s_g^* (%): pre-filter boundary at $s_{\text{loc}} = 0.3\%$ and 1% for two predicate terms (Theorem 5.1; 100 means pre-filtering wins for every s_g, 0 that it never does). #pre: queries for which the oracle / HVO choose Π_{pre}. Latencies in ms.

Data	Kernel	c_{scan} (ns)	s_g^*@0.3%	s_g^*@1%	#pre opt	#pre HVO	HVO mean	HVO P99	Oracle mean
SYN-128	Scalar FP32	64.4	10.2	0.0	242	271	1.16	8.71	1.08
SYN-128	NEON FP32	16.0	39.7	0.0	280	332	0.96	3.97	0.79
SYN-128	NEON INT8	3.3	100.0	0.0	298	340	0.91	3.46	0.71
SYN-768	Scalar FP32	564.4	1.5	0.0	198	221	3.18	37.66	4.15
SYN-768	NEON FP32	107.6	7.7	0.2	298	283	2.04	14.63	1.83
SYN-768	NEON INT8	25.2	33.0	0.9	382	393	1.49	8.08	1.22
ARXIV-384 (real)	Scalar FP32	240.3	6.9	0.3	299	349	4.36	54.00	3.62
ARXIV-384 (real)	NEON FP32	46.0	35.4	1.8	367	414	1.84	13.95	1.53
ARXIV-384 (real)	NEON INT8	8.7	100.0	8.3	408	467	1.18	5.79	0.95

- *Calibrator.* Without it, the raw sketch's heavier tail forces a wider interval (η from the raw estimator's training Q-error), and minimax routing becomes overly conservative.

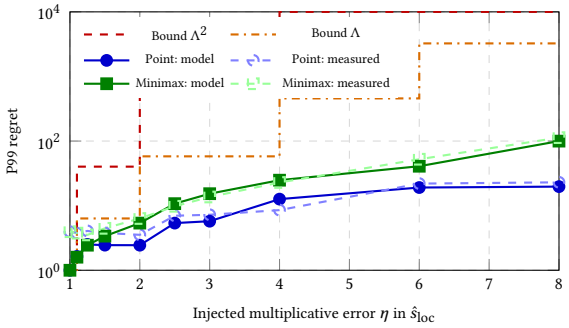


Figure 4: Regret on SYN-128 under injected estimation error ($\hat{s}_g = s_g$; 20 repetitions). Model regret uses the calibrated cost model at the true selectivities; measured regret uses the executed plans' latencies and includes the cost model's calibration error, which is why it exceeds 1 even at $\eta = 1$. Bounds are clipped at 10^4.

- *Interval width.* Widths between the 0.75- and 0.95-quantiles perform similarly; the 0.99-quantile scans too often.
- *Runtime guard.* The early-exit guard fired on 39, 3 and 7 test queries (SYN-128, SYN-768, ARXIV-384) and slightly *increased* mean latency. Its triggers come from the first, smallest round, where a low observed match rate is weak evidence. We report it as a negative result.

7.5 Regret versus estimation error

To test Theorems 6.2 and 6.3, we replace the estimate by $\hat{s}_{\text{loc}} = s_{\text{loc}}\,\eta^{\pm 1}$ with a random sign (an adversarial, interval-endpoint error), keep $\hat{s}_g = s_g$, and route the SYN-128 test queries 20 times per value of η (Figure 4). Across 300000 routed queries the model regret never exceeded the bound of Lemma 6.1 and Theorems 6.2–6.3 (0 violations). Minimax routing had zero regret on every query whose best plan was Π_{pre}.

The bounds are loose. With the measured $\gamma_{\text{fused}} = 2.42$, a single extra doubling round costs a factor $1 + 2^{\gamma} \approx 6.34$, which is the bound already at $\eta = 1.1$, while the observed P99 model regret is 1.63. The figure also shows the cost of conservatism. When every estimate sits at an endpoint of its interval, minimax routing is more conservative than point routing (P99 model regret 5.37 vs. 2.44 at $\eta = 2$). Under HPBS's actual error distribution, where most errors are small and a few are large, it removes the latency tail (Table 3).

8 Discussion and Limitations

Data. We have one real dataset. Its embeddings and metadata are real, but its predicates, like those of the synthetic benchmarks, are drawn at random rather than taken from a query log. The two synthetic benchmarks control correlation but share one generator. HVO's advantage was *larger* on real data than on synthetic data, which suggests the synthetic correlation is milder than real correlation. A second real domain (for example product catalogs or multimodal embeddings) and a real query workload would test this further.

One machine, one library. Absolute latencies and the fitted constants are specific to an Apple M4 and faiss's HNSW. The cost model's structure (staircases, a per-vector scan cost, a per-row predicate cost) carries over, but its constants must be recalibrated per machine, and the superlinear round cost we measure is a property of faiss's search implementation rather than of HNSW in general. We do not compare against `pgvector`, Milvus, DuckDB or ACORN as systems. Our global-selectivity routers reproduce their *routing policies* on identical plans, which isolates the optimizer from engine differences but does not measure those engines.

Planning overhead. HPBS's query path is C, but the predicate encoding and median-of-L logic are Python. At hundreds of microseconds of planning time against sub-millisecond plans, this matters. A fully native implementation inside a DBMS planner hook would reduce it, and $L = 1$ already cuts it by half or more at a small loss in tail accuracy.

Not evaluated. We have not evaluated updates, `GROUP BY` top-k (which changes the post-filtering horizon from k matches to k distinct groups), multi-way joins, two-hop fused traversal as in ACORN, or skewed foreign keys stress-tested in isolation. Higher-order attribute correlation inside a cell is not modeled.

9 Conclusion

Choosing a plan for filtered vector search requires knowing how far the search must travel before it meets k matches, and that distance is a local, correlation-dependent quantity. HVO estimates it with a compact sketch that never touches the index, prices plans with a cost model calibrated on the target hardware, and routes with a minimax rule that provably avoids the most expensive class of misroutes. On one million real arXiv papers, HPBS is the most accurate estimator in the tail and on rare local matches, and HVO lowers mean latency by 65% relative to tuned global-selectivity routing, with tail latency close to an oracle's. On two synthetic benchmarks the mean-latency gains are smaller (35% and 23%), and there HVO does not beat the threshold rule's P99 latency. Its recall is slightly below the oracle's on all three. The results come from one machine and plans built on faiss; validating them inside a production planner and on real query workloads is future work.

Declarations and Reproducibility

Funding and conflicts of interest: This research received no external funding. The authors declare no conflicts of interest.

Code and data: Implementation, data generator, experiment drivers, raw results and the scripts that generate every number, table and figure in this paper are available at https://github.com/sanmish4ds/HyperVectorOptimizer.

Author contributions: S. Mishra: conceptualization, methodology, formal analysis, software, writing (original draft). **D. Chukkapalli:** validation, investigation, visualization, writing (review and editing).

References

[1] C. Wei *et al.*, "AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3152–3165, 2020.

[2] Q. Zhang *et al.*, "VBase: Unifying online vector similarity search and relational queries via relaxed monotonicity," in *Proc. USENIX OSDI*, 2023, pp. 377–395.

Table 5: Ablations of HVO: mean and P99 latency (ms), plan accuracy (%), and P99 measured regret.

	SYN-128				SYN-768				ARXIV-384 (real)			
Variant	Mean	P99	Acc.	P99 reg.	Mean	P99	Acc.	P99 reg.	Mean	P99	Acc.	P99 reg.
HVO (robust, L=4, calibrated)	0.96	3.97	83.9	6.14	2.04	14.63	74.9	6.08	1.84	13.95	88.6	5.52
point routing (no interval)	1.06	12.05	86.9	5.89	2.15	18.00	82.2	4.76	2.01	15.66	91.0	4.67
without calibrator	1.34	4.10	54.5	21.96	2.97	14.27	34.7	15.71	2.46	15.79	46.1	22.65
L=1 partition	0.89	4.32	82.3	4.75	1.98	16.10	72.4	5.17	1.66	13.44	89.3	4.88
hardware-agnostic cost (distance counts)	1.92	14.00	68.5	26.17	4.54	47.63	74.0	8.41	4.78	68.15	87.6	8.81
+ runtime early-exit guard	1.02	4.36	83.9	6.93	2.05	14.63	74.9	6.17	1.87	13.95	88.6	6.09
interval q=0.50	0.96	4.97	87.0	5.88	2.18	18.50	79.5	5.26	1.75	12.74	90.6	4.35
interval q=0.75	0.91	3.97	87.0	5.82	2.06	17.20	78.5	4.55	1.82	13.95	90.1	5.20
interval q=0.95	1.00	4.91	81.1	6.19	1.97	14.48	70.5	6.36	1.86	13.72	86.5	6.59
interval q=0.99	1.05	3.71	74.1	7.30	2.27	13.63	57.3	9.88	2.03	14.31	79.7	13.34
planning time excluded	0.82	3.88	83.9	4.16	1.85	14.49	74.9	4.95	1.66	13.67	88.6	4.94

[3] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proc. ACM Manag. Data*, vol. 2, no. 3, 2024.

[4] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," arXiv:2602.11443, 2026.

[5] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of filtered approximate nearest neighbor search over the vector-scalar hybrid data," arXiv:2505.06501, 2025.

[6] P. Iff *et al.*, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," arXiv:2507.21989, 2025.

[7] J. Shi, Y. Cai, and W. Zheng, "Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study," arXiv:2509.07789, 2025.

[8] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2024. [Online]. Available: https://github.com/pgvector/pgvector

[9] DuckDB, "Vector similarity search extension (vss)," 2024. [Online]. Available: https://duckdb.org/docs/extensions/vss

[10] J. Wang *et al.*, "Milvus: A purpose-built vector data management system," in *Proc. ACM SIGMOD*, 2021, pp. 2614–2627.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[12] M. Douze *et al.*, "The Faiss library," arXiv:2401.08281, 2024.

[13] S. Gollapudi *et al.*, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf.*, 2023, pp. 3406–3416.

[14] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints," arXiv:2203.13601, 2022.

[15] W. Wu *et al.*, "HQANN: Efficient and robust similarity search for hybrid queries with structured and unstructured constraints," in *Proc. ACM CIKM*, 2022, pp. 4580–4584.

[16] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: Segment graph for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 1, 2024.

[17] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3256–3268, 2025.

[18] H. Kim, C. Lim, H. An, R. Sen, and K. Park, "Exqutor: Extended query optimizer for vector-augmented analytical queries," arXiv:2512.09695, 2025.

[19] P. G. Selinger *et al.*, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD*, 1979, pp. 23–34.

[20] Y. Matias, J. S. Vitter, and M. Wang, "Wavelet-based histograms for selectivity estimation," in *Proc. ACM SIGMOD*, 1998, pp. 448–459.

[21] D. Gunopulos, G. Kollios, V. J. Tsotras, and C. Domeniconi, "Approximating multi-dimensional aggregate range queries over real attributes," in *Proc. ACM SIGMOD*, 2000, pp. 463–474.

[22] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm," in *Proc. AofA*, 2007, pp. 127–146.

[23] K. Beyer, P. J. Haas, B. Reinwald, Y. Sismanis, and R. Gemulla, "On synopses for distinct-value estimation under multiset operations," in *Proc. ACM SIGMOD*, 2007, pp. 199–210.

[24] A. Kipf *et al.*, "Learned cardinalities: Estimating correlated joins with deep learning," in *Proc. CIDR*, 2019.

[25] Z. Yang *et al.*, "NeuroCard: One cardinality estimator for all tables," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 61–73, 2020.

[26] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig, "DeepDB: Learn from data, not from queries!" *Proc. VLDB Endow.*, vol. 13, no. 7, pp. 992–1005, 2020.

[27] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. ACM STOC*, 2002, pp. 380–388.

[28] R. Spring and A. Shrivastava, "A new unbiased and efficient class of LSH-based samplers and estimators for partition function computation in log-linear models," arXiv:1703.05160, 2017.

[29] H. Lan, S. Huang, Z. Bao, and R. Borovica-Gajic, "Cardinality estimation for similarity search on high-dimensional data objects: The impact of reference objects," *Proc. VLDB Endow.*, vol. 18, no. 3, pp. 544–556, 2024.

[30] "arxiv-abstracts-2021: metadata and abstracts of arXiv papers," Hugging Face dataset `gfissore/arxiv-abstracts-2021`, CC0 license, 2021. [Online]. Available: https://huggingface.co/datasets/gfissore/arxiv-abstracts-2021

[31] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.